

Real-Time Real Estate Investment Analysis

Xiangtao Fu, VS Praneeth, Sireesha Baratam

December 2025

Contents

Real-Time Real Estate Investment Analysis Web Application	3
Table of Contents	3
Overview	3
Features	4
Architecture	4
Tech Stack	4
Frontend	4
Backend	4
External APIs	5
Project Structure	5
Quick Start	5
1. Clone the Repository	5
2. Start the Backend	5
3. Start the Frontend	6
4. Access the Application	6
Configuration	6
API Keys	6
Backend Configuration (application.yml)	6
Frontend Configuration (.env)	6
API Documentation	7
Backend REST Endpoints	7
Other Docs	7
Development	8
Tests	8
Building for Production	8
Contributing	8
License	8
Credits	8
Contact	8
Contents	9
Introduction	9
System Requirements	9
Minimum	9
Software You Need	9

Installation Guide	9
Step 1: Install Prerequisites	9
Step 2: Clone the Repository	10
Step 3: Install Dependencies	10
Getting API Keys	11
1. RapidAPI Key (Zillow & Google API 31)	11
2. Google OAuth Client ID	11
3. Mapbox Access Token	11
Running the Application	12
Method 1: Using Scripts (Recommended)	12
Method 2: Manual Start	12
Accessing the Application	13
Using the Application	13
Landing Page & Login	13
Dashboard	13
Property Search	13
Investment Analysis	13
Property Comparison	13
Reports	13
Troubleshooting	13
Backend Problems	13
Frontend Issues	14
Authentication Issues	14
API Endpoints	14
Backend APIs (What's Implemented)	14
Frontend Integration (What's Actually Connected)	15
Quick API Tests	15
FAQ	15
General	15
Technical Questions	16
API Questions	16
Help	16
Contents	16
Architecture Overview	17
How It Fits Together	17
Component Responsibilities	17
Frontend Development	17
Project Structure	17
Environment Variables	19
Adding New Components	19
Backend Development	20
Project Structure	20
Key Classes	20
Configuration	22
Adding New Endpoints	22
API Reference	23
Backend Implementation Overview	23
APIs Connected to Frontend	23

Backend APIs Not Connected to Frontend	25
Data Models	25
Frontend (TypeScript)	25
Backend Models (Java)	27
Business Logic	27
Mortgage Calculation	27
Cash-on-Cash Return	28
Cap Rate	28
IRR Calculation (Newton-Raphson Method)	28
Testing	29
Frontend Testing	29
Backend Testing	29
Writing Tests	30
Deployment	31
Build for Production	31
Docker Deployment	31
Code Style	32
TypeScript/React	32
Java	32
Git Commits	32

Real-Time Real Estate Investment Analysis Web Application

Full-stack web app for real estate investment analysis. Search properties, run cashflow numbers, compare strategies (Rental, BRRRR, Flip, Wholesale), and export reports.

Table of Contents

- Overview
 - Features
 - Architecture
 - Tech Stack
 - Project Structure
 - Getting Started
 - Config
 - API Docs
 - Docs
 - Contributing
-

Overview

This app helps real estate investors analyze deals quickly. It pulls live data from Zillow, runs the numbers on different investment strategies, and shows everything on a map.

What it does: - Search properties anywhere in the US (Zillow data) - Run cashflow analysis, calculate

ROI, Cap Rate, IRR - Compare deals side by side - Support for Rental, BRRRR, Flip, and Wholesale strategies - Export reports - Google login

Features

Frontend - Dashboard to manage your saved properties - Search by city, zip, price range - Cashflow calculators for different strategies - Compare multiple properties - Mapbox maps with markers - PDF/HTML reports - Works on mobile - Google login

Backend - REST API - Zillow data (prices, Zestimate, rent estimates) - Google API 31 geocoding - Cashflow engine - CORS enabled

Strategies - Rental (buy & hold) - BRRRR - Fix & Flip - Wholesale

Architecture

Client Layer (React Frontend) - Components: AuthPage, Dashboard, SearchPage, PropertyComparison, ReportViewer - State: AuthContext, PropertiesContext - Communicates via HTTP/REST

Server Layer (Spring Boot) - Controllers: PropertyController, GeoController, CashflowController - Services: ZillowService, GoogleApi31Service, CashflowService

External Services - Zillow API (RapidAPI) - property data - Google API 31 (RapidAPI) - geocoding - Mapbox API (direct from frontend) - maps

Tech Stack

Frontend

- React 19.2.0
- TypeScript 4.9.5
- Mapbox GL JS 3.16.0, react-map-gl 8.1.0
- @react-oauth/google 0.12.2
- react-icons 5.5.0
- Create React App 5.0.1

Backend

- Spring Boot 3.2.0
- Java 17
- Maven 3.8+
- Jackson 2.15.x
- Java HttpClient (JDK 17)

External APIs

Backend implements: - Zillow API (RapidAPI) - Property search, details, Zestimate - Google API 31 (RapidAPI) - Geocoding, place search - Cashflow Engine (built-in) - Investment calculations

Frontend connects to: - Zillow via backend (/api/properties/*) - Property data - Cashflow via backend (/api/analysis/cashflow) - ROI calculations - Mapbox (direct) - Maps and geocoding - Google OAuth (direct) - User login

Note: The backend implements Google API 31 endpoints (/api/geo/*) for geocoding, but the frontend does not use them - it calls Mapbox directly instead. These endpoints remain available for future features.

Project Structure

Root: README.md, USER_GUIDE.md, DEVELOPER_GUIDE.md

Frontend (real-time-real-estate-analyzer/real-time-analyzer/): - src/App.tsx - Main component - src/components/ - AuthPage, Dashboard, SearchPage, PropertyMap, PropertyForm, Property-Comparison, etc. - src/context/ - AuthContext, PropertiesContext - src/utils/ - cashflowApi.ts, reportGenerator.ts

Backend (googlemapv2/): - pom.xml - Maven config - src/main/java/.../GoogleApi31Service.java - Geo service - src/main/java/.../CashflowController.java - REST API - src/main/java/.../CashflowService.java - Business logic - src/main/resources/application.yml - Config

Standalone Modules: - cashflow-calculator/ - Standalone calculator - zillow api/ - Zillow API client

Quick Start

You'll need Node.js 18+, Java 17+, Maven 3.8+, and some API keys (covered below).

1. Clone the Repository

```
git clone https://github.com/praneeth-vadrevu/Real-Time-Real-Estate-Investment-Analysis--Web-Ap
cd Real-Time-Real-Estate-Original
```

2. Start the Backend

```
cd googlemapv2
```

```
# Set environment variables (Windows PowerShell)
$env:RAPIDAPI_KEY = "your-rapidapi-key"
```

```
# Build and run
mvn clean install
mvn spring-boot:run
```

Backend will start at: <http://localhost:8080>

3. Start the Frontend

```
cd real-time-real-estate-analyzer/real-time-analyzer
```

```
# Install dependencies
```

```
npm install
```

```
# Set environment variables
```

```
# Create .env file with:
```

```
# REACT_APP_GOOGLE_CLIENT_ID=your-google-client-id
```

```
# REACT_APP_MAPBOX_TOKEN=your-mapbox-token
```

```
# Start development server
```

```
npm start
```

Frontend will start at: <http://localhost:3000>

4. Access the Application

Open your browser and navigate to <http://localhost:3000>

Configuration

API Keys

You need three keys: - **RAPIDAPI_KEY** - Get from RapidAPI, covers both Zillow and Google API 31 - **REACT_APP_GOOGLE_CLIENT_ID** - From Google Cloud Console - **REACT_APP_MAPBOX_TOKEN** - From Mapbox

Backend Configuration (application.yml)

```
server:
```

```
  port: 8080
```

```
rapidapi:
```

```
  key: ${RAPIDAPI_KEY}
```

```
  zillow:
```

```
    host: zillow-com1.p.rapidapi.com
```

```
  google:
```

```
    host: google-api31.p.rapidapi.com
```

```
cors:
```

```
  allowed-origins: http://localhost:3000
```

Frontend Configuration (.env)

```
REACT_APP_GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
```

```
REACT_APP_MAPBOX_TOKEN=pk.your-mapbox-token
```

```
REACT_APP_API_URL=http://localhost:8080
```

API Documentation

Backend REST Endpoints

Property Search

GET /api/zillow/search?location={location}&page={page}

Property Details

GET /api/zillow/property/{zpid}

Zestimate

GET /api/zillow/zestimate/{zpid}

Cashflow Analysis

POST /api/cashflow/calculate

Content-Type: application/json

Example request (sample values, not defaults):

```
{
  "purchasePrice": 300000,
  "downPaymentPercent": 20,
  "interestRate": 7.0,
  "loanTermYears": 30,
  "monthlyRent": 2500,
  "propertyTaxRate": 1.2,
  "insuranceAnnual": 1500,
  "maintenancePercent": 5,
  "vacancyPercent": 5,
  "managementPercent": 8
}
```

Geocoding (Backend only - not used by frontend)

GET /api/google/geocode?address={address}

For complete API documentation, see Backend Developer Guide.

Other Docs

- User Guide - How to install and use
- Developer Guide - Technical reference
- Backend README - Backend overview
- Cashflow Calculator - Calculator module

Development

Tests

Frontend:

```
cd real-time-real-estate-analyzer/real-time-analyzer  
npm test
```

Backend:

```
cd googlemapv2  
mvn test
```

Building for Production

Frontend:

```
npm run build
```

Backend:

```
mvn clean package -DskipTests  
java -jar target/googlemapv2-1.0.0.jar
```

Contributing

Fork it, create a branch, make changes, push, and open a PR.

License

MIT License. See LICENSE.

Credits

Built with Zillow API, Mapbox, Spring Boot, and React.

Contact

Authors: Xiangtao Fu, VS Praneeth, Sireesha Baratam

Repository: GitHub

Course: CS 682, Fall 2025

Last Updated: December 2025 # User Guide

Contents

- Introduction
 - System Requirements
 - Installation
 - API Keys
 - Running the App
 - How to Use It
 - Troubleshooting
 - FAQ
-

Introduction

This guide covers setup and usage for the Real-Time Real Estate Investment Analysis app. With it you can search US properties, run cashflow numbers, compare deals, and export reports.

System Requirements

Minimum

Windows 10+, macOS 10.15+, or Linux. 8 GB RAM, 2 GB disk space. Modern browser (Chrome, Firefox, Safari, Edge).

Software You Need

- Node.js 18+ from nodejs.org
 - Java 17+ from adoptium.net
 - Maven 3.8+ from maven.apache.org
 - Git from git-scm.com
-

Installation Guide

Step 1: Install Prerequisites

Windows

1. **Install Node.js:**
 - Download from nodejs.org
 - Run the installer and follow prompts
 - Verify: `node --version`
2. **Install Java 17:**
 - Download from adoptium.net
 - Run the installer
 - Set `JAVA_HOME` environment variable
 - Verify: `java -version`
3. **Install Maven:**

- Download from maven.apache.org
- Extract to C:\Program Files\Apache\maven
- Add to PATH: C:\Program Files\Apache\maven\bin
- Verify: `mvn -version`

macOS

```
# Install Homebrew (if not installed)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install dependencies
brew install node@18
brew install openjdk@17
brew install maven

# Verify installations
node --version
java -version
mvn -version
```

Linux (Ubuntu/Debian)

```
# Update package list
sudo apt update

# Install Node.js 18
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Install Java 17
sudo apt install -y openjdk-17-jdk

# Install Maven
sudo apt install -y maven

# Verify installations
node --version
java -version
mvn -version
```

Step 2: Clone the Repository

```
git clone https://github.com/praneeth-vadrevu/Real-Time-Real-Estate-Investment-Analysis--Web-Ap
cd Real-Time-Real-Estate-Original
```

Step 3: Install Dependencies

Backend:

```
cd googlemapv2
mvn clean install
```

Frontend:

```
cd real-time-real-estate-analyzer/real-time-analyzer
npm install
```

Getting API Keys

You need a few API keys. All have free tiers.

1. RapidAPI Key (Zillow & Google API 31)

This single key provides access to both Zillow and Google API 31 services.

1. Go to RapidAPI
2. Create a free account
3. Subscribe to these APIs:
 - Zillow API (Free tier available)
 - Google API 31 (Free tier available)
4. Copy your RapidAPI key from the dashboard

2. Google OAuth Client ID

Required for user authentication.

1. Go to Google Cloud Console
2. Create a new project or select existing
3. Navigate to “APIs & Services” > “Credentials”
4. Click “Create Credentials” > “OAuth client ID”
5. Configure consent screen if prompted
6. Select “Web application”
7. Add authorized JavaScript origins:
 - `http://localhost:3000`
8. Add authorized redirect URIs:
 - `http://localhost:3000`
9. Copy the Client ID

3. Mapbox Access Token

Required for interactive maps.

1. Go to Mapbox
 2. Create a free account
 3. Navigate to “Access tokens”
 4. Copy your default public token or create a new one
-

Running the Application

Method 1: Using Scripts (Recommended)

Windows:

```
# Start Backend  
cd googlemapv2  
.\start-backend.bat  
  
# In a new terminal, start Frontend  
cd real-time-real-estate-analyzer\real-time-analyzer  
npm start
```

macOS/Linux:

```
# Start Backend  
cd googlemapv2  
./start-backend.sh  
  
# In a new terminal, start Frontend  
cd real-time-real-estate-analyzer/real-time-analyzer  
npm start
```

Method 2: Manual Start

Start Backend:

```
cd googlemapv2  
  
# Set API key (Windows PowerShell)  
$env:RAPIDAPI_KEY = "your-rapidapi-key-here"  
  
# Or Linux/macOS  
export RAPIDAPI_KEY="your-rapidapi-key-here"  
  
# Run  
mvn spring-boot:run
```

Start Frontend:

```
cd real-time-real-estate-analyzer/real-time-analyzer  
  
# Create .env file  
echo "REACT_APP_GOOGLE_CLIENT_ID=your-google-client-id" > .env  
echo "REACT_APP_MAPBOX_TOKEN=your-mapbox-token" >> .env  
  
# Run  
npm start
```

Accessing the Application

Once both services are running:

- **Frontend:** `http://localhost:3000`
 - **Backend API:** `http://localhost:8080`
-

Using the Application

Landing Page & Login

When you open the app you see login options. Click “Sign in with Google” for full access, or “Continue as Guest” for limited features.

Dashboard

The dashboard shows your saved properties. Use the sidebar to switch strategies, the main area to view deals, and the “+” button to add new ones.

Property Search

Go to “Search Properties”, type a city/state/ZIP, and results show up on the map. Click any property for details, then “Save” to add it to your portfolio.

Investment Analysis

Pick a property from your portfolio. The app calculates monthly cashflow, cash-on-cash, cap rate, and IRR. You can tweak the inputs (down payment, rate, etc.) and recalculate.

Property Comparison

Select multiple properties and hit “Compare” to see them side by side. Export as needed.

Reports

Select a property, click “Generate Report”, pick PDF or HTML, and download.

Troubleshooting

Backend Problems

Problem: `mvn: command not found`

```
# Add Maven to PATH
# Windows: Add C:\Program Files\Apache\maven\bin to system PATH
# macOS/Linux: Add to ~/.bashrc or ~/.zshrc
export PATH=$PATH:/path/to/maven/bin
```

Problem: Port 8080 already in use

```
# Windows
netstat -ano | findstr :8080
taskkill /PID <pid> /F
```

```
# macOS/Linux
lsof -i :8080
kill -9 <pid>
```

Problem: RAPIDAPI_KEY not set

Ensure environment variable is set before running

Windows PowerShell

```
$env:RAPIDAPI_KEY = "your-key"
```

Linux/macOS

```
export RAPIDAPI_KEY="your-key"
```

Frontend Issues

Problem: npm: command not found - Reinstall Node.js from nodejs.org

Problem: Module not found errors

```
rm -rf node_modules
npm install
```

Problem: Map not loading - Verify REACT_APP_MAPBOX_TOKEN in .env file - Check browser console for errors - Ensure Mapbox token has correct permissions

Authentication Issues

Problem: Google login not working - Verify REACT_APP_GOOGLE_CLIENT_ID in .env - Check Google Cloud Console for correct redirect URIs - Ensure OAuth consent screen is configured

API Endpoints

Backend APIs (What's Implemented)

The Spring Boot backend exposes these endpoints:

Zillow API Endpoints (via RapidAPI): - GET /api/properties/search - Search properties by location - GET /api/properties/{zpid} - Get property details by ZPID

Cashflow Analysis: - POST /api/analysis/cashflow - Calculate investment metrics (NOI, Cap Rate, IRR, etc.)

Google API 31 Endpoints (backend only, not used by frontend): - GET /api/geo/text - Text-based place search - GET /api/geo/circle - Radius-based location search - POST /api/geo/raw - Raw query passthrough

Frontend Integration (What's Actually Connected)

The React frontend currently calls these APIs:

Backend API Calls:

- `/api/properties/search` - Called in `SearchPage.tsx`, `PropertySearch.tsx` for Zillow property search
- `/api/properties/{zpid}` - Called in `PropertyForm.tsx` for Zillow property details
- `/api/analysis/cashflow` - Called in `cashflowApi.ts` for investment calculations

Direct External Calls (not through backend):

- Mapbox API - Called in `PropertyMap.tsx` for map display and geocoding
- Google OAuth - Called in `AuthContext.tsx` for user login

Not Connected: The backend implements Google API 31 geo endpoints (`/api/geo/*`) for geocoding and place search, but the frontend uses Mapbox directly instead. These endpoints remain available for future features like nearby amenities or walkability scoring.

Quick API Tests

Test the active backend APIs (example values shown):

```
# Test property search
```

```
curl "http://localhost:8080/api/properties/search?location=Santa%20Monica,%20CA&status=for_sale"
```

```
# Test property details (example ZPID)
```

```
curl "http://localhost:8080/api/properties/20479916"
```

```
# Test cashflow analysis (example values, not defaults)
```

```
curl -X POST "http://localhost:8080/api/analysis/cashflow" \
  -H "Content-Type: application/json" \
  -d '{"offerPrice": 300000, "grossRentsAnnual": 36000, "firstPrincipal": 240000, "firstRateAnnual": 0.05}'
```

Test the backend-only Google Geo APIs (not used by frontend):

```
# Text-based place search
```

```
curl "http://localhost:8080/api/geo/text?text=coffee&city=Santa%20Monica&state=CA"
```

```
# Circular range search
```

```
curl "http://localhost:8080/api/geo/circle?lat=34.0195&lon=-118.4912&radius=5000&text=restaurant"
```

FAQ

General

Q: Is the application free to use? A: The application itself is open source. However, the external APIs (Zillow, Google API 31, Mapbox) have free tiers with usage limits. For heavy use, paid plans may be required.

Q: Can I use this for commercial purposes? A: Please check the API provider terms of service. Zillow API has specific restrictions on commercial use.

Q: Does the application store my data? A: Currently, property data is stored locally in your browser. No data is sent to external servers beyond the API calls.

Technical Questions

Q: Which browsers are supported? A: Chrome, Firefox, Safari, and Edge (latest versions). Internet Explorer is not supported.

Q: Can I run this on a server? A: Yes, both frontend and backend can be deployed to cloud services. See DEVELOPER_GUIDE.md for deployment instructions.

Q: How accurate is the investment analysis? A: The calculations are based on industry-standard formulas. However, actual results may vary based on market conditions, property-specific factors, and other variables.

API Questions

Q: Why am I getting API errors? A: Common causes: - Invalid or expired API key - API rate limit exceeded - Network connectivity issues

Q: What's the API rate limit? A: Free tier limits vary by provider: - Zillow API: ~100 requests/month - Google API 31: ~100 requests/month - Mapbox: 50,000 map loads/month

Help

Check the GitHub Issues or the Developer Guide.

December 2025 # Developer Guide

Contents

- Architecture
 - Frontend
 - Backend
 - API Reference
 - Data Models
 - Business Logic
 - Testing
 - Deployment
 - Code Style
-

Architecture Overview

How It Fits Together

Browser (React App) - Components: UI elements - Context Providers: Global state (Auth, Properties) - Utils: API calls, helpers

Spring Boot Backend - Controllers: Handle HTTP requests - Services: Business logic - Models: DTOs

External APIs - Zillow API, Google API 31, Mapbox API

Component Responsibilities

Frontend: - Components - UI rendering and user interaction - Context - Global state management - Utils - API calls and helper functions

Backend: - Controllers - HTTP request handling - Services - Business logic implementation - Models - Data structures and DTOs

Frontend Development

Project Structure

Frontend files are in `real-time-real-estate-analyzer/real-time-analyzer/src/`:

Main files: App.tsx, App.css, index.tsx

components/: AuthPage.tsx, Dashboard.tsx, SearchPage.tsx, PropertyMap.tsx, PropertyForm.tsx, PropertyComparison.tsx, PropertyReportViewer.tsx, PurchaseCriteriaForm.tsx, Navbar.tsx, Sidebar.tsx, MainContent.tsx, Footer.tsx, PropertyPopup.tsx

context/: AuthContext.tsx, PropertiesContext.tsx

utils/: cashflowApi.ts, reportGenerator.ts ### Key Components

App.tsx Main application component that handles routing and page state.

```
// Key state management
const [activePage, setActivePage] = useState("my-properties");
const [activeView, setActiveView] = useState<string | null>('auth');
const [showPropertyForm, setShowPropertyForm] = useState(false);

// Navigation handler
const handleNavigate = (page: string) => {
  setActivePage(page);
  if (page === 'home' || page === 'my-properties') {
    setActiveView('dashboard');
  } else if (page === 'search-properties') {
    setActiveView('search-properties');
  }
}
```

```
// ...
};
```

AuthContext.tsx Manages user authentication state using Google OAuth.

```
interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isGuest: boolean;
  isLoading: boolean;
  login: (credential: string) => void;
  loginAsGuest: () => void;
  logout: () => void;
}
```

PropertiesContext.tsx Manages the global properties state across all components.

```
interface PropertiesContextType {
  properties: Property[];
  addProperty: (property: Property) => void;
  removeProperty: (id: string) => void;
  updateProperty: (id: string, data: Partial<Property>) => void;
  getPropertyByZpid: (zpid: string) => Property | undefined;
}
```

cashflowApi.ts API client for backend communication. This module handles the cashflow analysis API calls.

```
// Cashflow analysis endpoint
const CASHFLOW_API_URL = 'http://localhost:8080/api/analysis/cashflow';

// Main analysis function
export async function analyzeCashflow(request: CashflowRequest): Promise<CashflowResponse> {
  const response = await fetch(CASHFLOW_API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(request),
  });

  if (!response.ok) {
    throw new Error(`Failed to analyze cashflow: ${response.status}`);
  }

  return response.json();
}

// Helper to convert PropertyForm data to API request format
export function mapPropertyDataToCashflowRequest(formData: PropertyData): CashflowRequest {
  return {
```

```

    offerPrice: formData.offerPrice,
    grossRentsAnnual: formData.grossRents,
    vacancyRate: formData.vacancyRate / 100,
    managementRate: formData.managementRate / 100,
    // ... other field mappings
  };
}

```

Note: Property search (`/api/properties/search`) and property details (`/api/properties/{zpid}`) API calls are made directly in components (`SearchPage.tsx`, `PropertySearch.tsx`, `PropertyForm.tsx`) rather than through this utility module.

Environment Variables

Create a `.env` file in the frontend root:

```

REACT_APP_GOOGLE_CLIENT_ID=your-google-client-id.apps.googleusercontent.com
REACT_APP_MAPBOX_TOKEN=pk.your-mapbox-token
REACT_APP_API_URL=http://localhost:8080

```

Adding New Components

1. Create component file in `src/components/`
2. Define TypeScript interfaces for props
3. Implement component with proper typing
4. Export and import where needed

Example:

```

// src/components/NewComponent.tsx
import React from 'react';

interface NewComponentProps {
  title: string;
  data: SomeDataType;
  onAction: (id: string) => void;
}

const NewComponent: React.FC<NewComponentProps> = ({ title, data, onAction }) => {
  return (
    <div className="new-component">
      <h2>{title}</h2>
      { /* Component content */ }
    </div>
  );
};

export default NewComponent;

```

Backend Development

Project Structure

```
googlemapv2/
src/
  main/
    java/com/example/googlemapv2/
      GoogleApi31Service.java      # Main service class
      CashflowController.java     # REST controller
      CashflowService.java        # Business logic
      CashflowRequest.java        # Request DTO
      CashflowResponse.java       # Response DTO

    resources/
      application.yml              # Configuration

  test/
    java/com/example/googlemapv2/
      CashflowServiceTest.java

pom.xml
README.md
USER_GUIDE.md
DEVELOPER_GUIDE.md
```

Key Classes

GoogleApi31Service.java Service class for Google API 31 integration (not currently used by frontend).

```
@Service
public class GoogleApi31Service {
    // Text-based search
    public JsonNode byText(String text, String place, String city,
                           String state, String country, String postcode);

    // Circular range search
    public JsonNode byCircle(double latitude, double longitude,
                             String text, int radiusMeters);

    // Raw query passthrough
    public JsonNode query(Map<String, Object> params);
}
```

GeoController.java REST controller exposing Google Geo API endpoints (not currently used by frontend).

```
@RestController
@RequestMapping("/api/geo")
```

```

public class GeoController {
    @GetMapping("/text")    // Text-based place search
    @GetMapping("/circle") // Radius-based search
    @PostMapping("/raw")    // Raw API passthrough
}

```

CashflowController.java (Used by Frontend) Handles investment cashflow analysis.

```

@RestController
@RequestMapping("/api/analysis")
public class CashflowController {

    private final CashflowService svc = new CashflowService();

    @PostMapping("/cashflow")
    public CashflowResponse analyze(@RequestBody CashflowRequest req) {
        return svc.analyze(req);
    }
}

```

PropertyController.java (Used by Frontend) Handles property search and details (implemented in `com.example.realestate.controller`).

```

@RestController
@RequestMapping("/api/properties")
public class PropertyController {
    @GetMapping("/search")    // Search properties by location
    @GetMapping("/{zpid}")    // Get property details by ZPID
}

```

Note: PropertyController source code is compiled but not included in this repository. It interfaces with the Zillow API via RapidAPI.

CashflowService.java Business logic for investment calculations.

```

@Service
public class CashflowService {

    public CashflowResponse calculate(CashflowRequest request) {
        // Calculate mortgage payment
        double monthlyPayment = calculateMortgage(
            request.getPurchasePrice(),
            request.getDownPaymentPercent(),
            request.getInterestRate(),
            request.getLoanTermYears()
        );

        // Calculate expenses
        double monthlyExpenses = calculateExpenses(request);
    }
}

```

```

        // Calculate cashflow
        double monthlyCashflow = request.getMonthlyRent() - monthlyPayment - monthlyExpenses;

        // Calculate ROI metrics
        double cashOnCash = calculateCashOnCash(monthlyCashflow, request);
        double capRate = calculateCapRate(request);
        double irr = calculateIRR(request);

        return new CashflowResponse(/* ... */);
    }
}

```

Configuration

application.yml:

```

server:
  port: 8080

spring:
  application:
    name: real-estate-backend

rapidapi:
  key: ${RAPIDAPI_KEY:your-default-key}
  zillow:
    host: zillow-com1.p.rapidapi.com
  google:
    host: google-api31.p.rapidapi.com

logging:
  level:
    com.example.googlemapv2: DEBUG

```

Adding New Endpoints

1. Create controller class or add to existing
2. Define service class for business logic
3. Create request/response DTOs
4. Add appropriate annotations

Example:

```

// New endpoint in controller
@GetMapping("/new-feature")
public ResponseEntity<NewFeatureResponse> newFeature(
    @RequestParam String param) {
    NewFeatureResponse response = newFeatureService.process(param);
}

```

```
    return ResponseEntity.ok(response);  
}
```

API Reference

Backend Implementation Overview

The backend provides three categories of APIs:

1. **Zillow API (via RapidAPI)** - Property search and details
2. **Cashflow Analysis** - Investment calculations (built-in)
3. **Google API 31 (via RapidAPI)** - Geocoding and place search

APIs Connected to Frontend

These are called by the React app:

Property Search (Zillow)

GET /api/properties/search

Parameters: - `location` (string, required) - City, state, or ZIP code - `status` (string, optional) - Listing status: `for_sale`, `for_rent`, `sold` - `page` (integer, optional) - Page number, default 1

Frontend Usage: `SearchPage.tsx`, `PropertySearch.tsx`

Response:

```
[  
  {  
    "propertyId": "20479916",  
    "address": "123 Main St, Santa Monica, CA 90401",  
    "streetAddress": "123 Main St",  
    "city": "Santa Monica",  
    "state": "CA",  
    "zip": "90401",  
    "county": "Los Angeles",  
    "countyFIPS": "06037",  
    "listPrice": 350000,  
    "bedrooms": 3,  
    "bathrooms": 2,  
    "livingArea": 1500  
  }  
]
```

Property Details (Zillow)

GET /api/properties/{zpid}

Called from: `PropertyForm.tsx`

Response: Full property details including coordinates, images, tax info, etc.

Cashflow Analysis

POST /api/analysis/cashflow

Content-Type: application/json

Frontend Usage: cashflowApi.ts analyzeCashflow()

Request Body:

```
{
  "address": "123 Main St",
  "city": "Santa Monica",
  "state": "CA",
  "zip": "90401",
  "fmv": 350000,
  "offerPrice": 300000,
  "annualAppreciation": 0.03,
  "grossRentsAnnual": 36000,
  "numberOfUnits": 1,
  "vacancyRate": 0.05,
  "managementRate": 0.08,
  "repairsRate": 0.05,
  "propertyTaxes": 3600,
  "insurance": 1500,
  "firstPrincipal": 240000,
  "firstRateAnnual": 0.07,
  "firstAmortYears": 30,
  "holdYears": 10,
  "rentGrowth": 0.03,
  "expenseGrowth": 0.03,
  "exitCostRate": 0.06
}
```

Response:

```
{
  "summary": {
    "cashToClose": 60000,
    "noiY1": 24000,
    "cashOnCashY1": 8.57,
    "capRateFMVY1": 6.86,
    "dscrY1": 1.25,
    "irr": 12.3,
    "equityMultiple": 2.1
  },
  "projection": [
    {
      "year": 1,
      "totalIncome": 36000,
      "vacancyLoss": 1800,
      "egi": 34200,

```

```

    "totalExpenses": 10200,
    "noi": 24000,
    "debtService": 19160,
    "cashFlowBeforeTax": 4840
  }
]
}

```

Backend APIs Not Connected to Frontend

These endpoints are implemented but the frontend uses Mapbox directly for maps/geocoding instead.

Google API 31 - Text Search

GET /api/geo/text

Parameters (all optional): - **text** - Search keyword - **place** - Place name - **city** - City name - **state** - State code - **country** - Country name - **postcode** - ZIP/postal code

Example:

```
curl "http://localhost:8080/api/geo/text?text=restaurant&city=Santa%20Monica&state=CA"
```

Circle Radius Search

GET /api/geo/circle

Parameters: - **lat** (double, required) - Latitude of center - **lon** (double, required) - Longitude of center - **radius** (int, optional) - Radius in meters, default 1000 - **text** (string, optional) - Search keyword

Example:

```
curl "http://localhost:8080/api/geo/circle?lat=34.0195&lon=-118.4912&radius=5000&text=coffee"
```

Raw Query Passthrough

POST /api/geo/raw

Content-Type: application/json

Request Body: Any valid Google API 31 query parameters.

Note: These endpoints can be used for future features like nearby amenities search, walkability scoring, or location intelligence.

Data Models

Frontend (TypeScript)

```

// Property model
interface Property {

```

```

    zipid: string;
    address: string;
    price: number;
    bedrooms: number;
    bathrooms: number;
    livingArea: number;
    lotSize?: number;
    yearBuilt?: number;
    propertyType: string;
    latitude: number;
    longitude: number;
    zestimate?: number;
    rentZestimate?: number;
    images?: string[];
    investmentStrategy: 'rental' | 'brrrr' | 'flip' | 'wholesale';
}

```

```

// Cashflow request
interface CashflowRequest {
    purchasePrice: number;
    downPaymentPercent: number;
    interestRate: number;
    loanTermYears: number;
    monthlyRent: number;
    propertyTaxRate: number;
    insuranceAnnual: number;
    maintenancePercent: number;
    vacancyPercent: number;
    managementPercent: number;
    closingCostPercent?: number;
    rehabCost?: number;
}

```

```

// Cashflow response
interface CashflowResponse {
    monthlyMortgage: number;
    monthlyExpenses: number;
    monthlyCashflow: number;
    annualCashflow: number;
    cashOnCashReturn: number;
    capRate: number;
    irr: number;
    totalCashNeeded: number;
}

```

```

// User model
interface User {
    id: string;
}

```

```

    email: string;
    name: string;
    picture?: string;
}

```

Backend Models (Java)

```

// CashflowRequest.java
public class CashflowRequest {
    private double purchasePrice;
    private double downPaymentPercent;
    private double interestRate;
    private int loanTermYears;
    private double monthlyRent;
    private double propertyTaxRate;
    private double insuranceAnnual;
    private double maintenancePercent;
    private double vacancyPercent;
    private double managementPercent;
    private double closingCostPercent;
    private double rehabCost;

    // Getters and setters
}

```

```

// CashflowResponse.java
public class CashflowResponse {
    private double monthlyMortgage;
    private double monthlyExpenses;
    private double monthlyCashflow;
    private double annualCashflow;
    private double cashOnCashReturn;
    private double capRate;
    private double irr;
    private double totalCashNeeded;

    // Getters and setters
}

```

Business Logic

Mortgage Calculation

```

/**
 * Monthly mortgage payment using amortization formula
 *  $M = P * [r(1+r)^n] / [(1+r)^n - 1]$ 
 */

```

```

private double calculateMortgage(double principal, double downPaymentPct,
                                double annualRate, int years) {
    double loanAmount = principal * (1 - downPaymentPct / 100);
    double monthlyRate = annualRate / 100 / 12;
    int totalPayments = years * 12;

    if (monthlyRate == 0) return loanAmount / totalPayments;

    return loanAmount *
        (monthlyRate * Math.pow(1 + monthlyRate, totalPayments)) /
        (Math.pow(1 + monthlyRate, totalPayments) - 1);
}

```

Cash-on-Cash Return

```

/**
 * Cash-on-Cash Return = Annual Cashflow / Total Cash Invested
 */
private double calculateCashOnCash(double annualCashflow, CashflowRequest request) {
    double downPayment = request.getPurchasePrice() *
        (request.getDownPaymentPercent() / 100);
    double closingCosts = request.getPurchasePrice() *
        (request.getClosingCostPercent() / 100);
    double totalCashInvested = downPayment + closingCosts + request.getRehabCost();

    return (annualCashflow / totalCashInvested) * 100;
}

```

Cap Rate

```

/**
 * Cap Rate = NOI / Property Value
 * NOI = Annual Rent - Annual Operating Expenses (excluding mortgage)
 */
private double calculateCapRate(CashflowRequest request) {
    double annualRent = request.getMonthlyRent() * 12;
    double annualExpenses = calculateAnnualExpenses(request);
    double noi = annualRent - annualExpenses;

    return (noi / request.getPurchasePrice()) * 100;
}

```

IRR Calculation (Newton-Raphson Method)

```

/**
 * Internal Rate of Return using iterative Newton-Raphson method
 */
private double calculateIRR(double[] cashflows) {
    double irr = 0.1; // Initial guess 10%
}

```

```

double tolerance = 0.0001;
int maxIterations = 100;

for (int i = 0; i < maxIterations; i++) {
    double npv = 0;
    double derivativeNpv = 0;

    for (int t = 0; t < cashflows.length; t++) {
        npv += cashflows[t] / Math.pow(1 + irr, t);
        if (t > 0) {
            derivativeNpv -= t * cashflows[t] / Math.pow(1 + irr, t + 1);
        }
    }

    double newIrr = irr - npv / derivativeNpv;

    if (Math.abs(newIrr - irr) < tolerance) {
        return newIrr * 100;
    }

    irr = newIrr;
}

return irr * 100;
}

```

Testing

Frontend Testing

```
cd real-time-real-estate-analyzer/real-time-analyzer
```

```
# Run all tests
```

```
npm test
```

```
# Run tests with coverage
```

```
npm test -- --coverage
```

```
# Run specific test file
```

```
npm test -- AuthPage.test.tsx
```

Backend Testing

```
cd googlemapv2
```

```
# Run all tests
```

```
mvn test
```

```
# Run specific test class
mvn test -Dtest=CashflowServiceTest
```

```
# Run with coverage
mvn test jacoco:report
```

Writing Tests

Frontend (Jest + React Testing Library):

```
import { render, screen, fireEvent } from '@testing-library/react';
import PropertyForm from './PropertyForm';

describe('PropertyForm', () => {
  it('renders form fields', () => {
    render(<PropertyForm onSubmit={jest.fn()} />);
    expect(screen.getByLabelText('Purchase Price')).toBeInTheDocument();
    expect(screen.getByLabelText('Monthly Rent')).toBeInTheDocument();
  });

  it('submits form with correct values', () => {
    const handleSubmit = jest.fn();
    render(<PropertyForm onSubmit={handleSubmit} />);

    fireEvent.change(screen.getByLabelText('Purchase Price'), {
      target: { value: '300000' }
    });
    fireEvent.click(screen.getByText('Calculate'));

    expect(handleSubmit).toHaveBeenCalledWith(
      expect.objectContaining({ purchasePrice: 300000 })
    );
  });
});
```

Backend (JUnit 5):

```
@SpringBootTest
class CashflowServiceTest {

    @Autowired
    private CashflowService cashflowService;

    @Test
    void shouldCalculateCorrectMortgagePayment() {
        CashflowRequest request = new CashflowRequest();
        request.setPurchasePrice(300000);
        request.setDownPaymentPercent(20);
    }
}
```

```

        request.setInterestRate(7.0);
        request.setLoanTermYears(30);

        CashflowResponse response = cashflowService.calculate(request);

        assertEquals(1596.73, response.getMonthlyMortgage(), 0.01);
    }

    @Test
    void shouldCalculatePositiveCashflow() {
        CashflowRequest request = createSampleRequest();
        request.setMonthlyRent(2500);

        CashflowResponse response = cashflowService.calculate(request);

        assertTrue(response.getMonthlyCashflow() > 0);
    }
}

```

Deployment

Build for Production

Frontend:

```

cd real-time-real-estate-analyzer/real-time-analyzer
npm run build

```

Backend:

```

cd googlemapv2
mvn clean package -DskipTests

```

Docker Deployment

Dockerfile (Frontend):

```

FROM node:18-alpine as build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Dockerfile (Backend):

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/googlemapv2-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.yml:

```
version: '3.8'
services:
  frontend:
    build: ./real-time-real-estate-analyzer/real-time-analyzer
    ports:
      - "3000:80"
    environment:
      - REACT_APP_API_URL=http://backend:8080

  backend:
    build: ./googlemapv2
    ports:
      - "8080:8080"
    environment:
      - RAPIDAPI_KEY=${RAPIDAPI_KEY}
```

Code Style

TypeScript/React

- Use functional components with hooks
- Define interfaces for all props
- Use descriptive variable names
- Keep components focused and small
- Use CSS modules or styled-components

Java

- Follow Google Java Style Guide
- Use meaningful class and method names
- Document public methods with Javadoc
- Keep methods short (< 30 lines)
- Use dependency injection

Git Commits

```
feat: Add property comparison feature
fix: Resolve map marker positioning issue
docs: Update API documentation
test: Add unit tests for cashflow service
refactor: Simplify mortgage calculation logic
```

December 2025
